

AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 9.1: Agentic AI for Research Workflows

Zhigang Feng

Spring 2026

Topic 9.1 Agenda

1. **Framing** — what is agentic AI, autonomy spectrum
2. **How Agents Invoke Tools** — JSON tool calls under the hood
3. **Setup** — Git workflow and durable infrastructure

Arc: T1 Framing → T2 Under the Hood → T3 Homotopy → T4 Project Org → T5 Mistakes/Context → T6 Wrap.

Framing

From chat tools to durable research workflows

Why Economists Should Care

The bottleneck in applied research is often not ideas. It is execution.

- Turning a good question into clean code, diagnostics, tables, and figures is slow
- Many research tasks are repetitive: reshape data, rerun specifications, trace sources, refactor scripts, update plots
- Agentic tools compress the time from *idea* to *first defensible result*
- That raises the return to the skills economists uniquely provide: question design, institutional knowledge, validation, and judgment

The promise is not “AI does research for you.” The promise is faster implementation and faster iteration around a question you still have to define and defend.

Four Research Patterns

Pattern	Representative task	Main risk if you use the wrong workflow
Reusable skill	Referee-style paper review	Generic comments, hallucinated quotes, missing quality gate
Empirical pipeline	MEPS harmonization and figures	Wrong crosswalk, dropped survey weights, silent validation failure
Public-web dataset	Fellows metadata with provenance (field-level source trace)	Weak sources, hidden missingness, no audit trail
Full-lifecycle col-laborator	Research memo → pipeline → critique → revision	Polished output built on weak framing or claims discipline

This lecture is about terminal-agent workflows. Claude Code, Codex CLI, Gemini CLI, Aider, and Cursor are examples; the durable lesson is how economists structure files, tools, validation, and judgment.

Where RAG fits. Agentic RAG (Lec08 T4b) differs from single-shot RAG: the agent decides *dynamically*, via the same JSON tool-call mechanism, whether and when to retrieve. RAG is one tool among many — alongside code execution, file I/O, and web fetch — that the agent can compass.

What Remains Human Across the Four Patterns

You still own

- the research question
- the success criteria
- the coding and source rules
- the validation target
- the final interpretation

The agent accelerates

- repetitive execution
- file handling and refactoring
- promptable decomposition
- reruns after fixes
- draft diagnostics and reports

Why this matters for economists: the gain is not that judgment disappears. The gain is that disciplined execution becomes cheaper.

What Modern Web Chat Already Does Well

As of April 2026, a strong web-chat product is already useful for economists.

- Brainstorming model setups, empirical strategies, review rubrics, and robustness checks
- Summarizing papers, referee reports, and lecture notes
- Inspecting uploaded files and explaining code snippets
- Sometimes browsing the web or running limited sandboxed tools
- Helping you draft a specification before you open your editor

So the weak baseline is not “a browser cannot do anything.” The weak baseline is a *single chat session without a durable harness*.

What Still Breaks Without a Project Harness

A single web-chat session often fails at the exact points where research projects become real.

1. **Durable context is weak.** Yesterday's conventions and corrections are not reliably embedded in today's work
2. **Reusable workflow is weak.** Your validation checklist lives in prompts, not in inspectable files
3. **Controlled execution is weak.** The model is not operating inside your actual project tree with your actual scripts and outputs
4. **Audit trail and provenance are weak.** It is hard to inspect which files, commands, checks, and sources produced the result

That is the real contrast. Agents are not magic because they are “smarter.” They are useful because they can operate inside a structured project with files, tools, and repeatable workflows.

What Makes a Workflow Agentic Under the Hood?

An agentic workflow is not just text generation. It is a control loop wrapped around tools, files, and checks.



- **Orchestrator:** the loop deciding what to do next
- **Memory:** saved project state on disk, not just chat history
- **Verifier:** tests, benchmarks, or review checks deciding whether output survives

How Agents Invoke Tools: JSON Schemas

Lec07 T3a explained that an LLM can emit structured JSON via schema-constrained decoding. In a terminal agent, every tool call uses that mechanism.

Example. When the agent decides to read a file, it emits:

```
{"tool": "Read", "args": {"file_path": "aiyagari_v0.py"}}
```

The harness parses the JSON, executes Read, and feeds the file content back as the next user message. The agent reads the result and decides the next action.

- **Loop.** Model output (JSON) → harness execution → tool result → next model turn. Continues until the model emits a “done” signal or hits a stop condition.
- **Schema is the contract.** Each tool advertises an argument schema; outputs that fail the schema are rejected.
- **Same brain.** There is no separate “planner” — the same transformer that writes code also decides to call tools.
- **Key implication.** Every action the agent takes is a *next-token prediction* constrained by the available tool schema.

How Should Economists Evaluate an Agentic Workflow?

Criterion	Question to ask
Correctness	Did it compute, extract, or summarize the right object?
Reproducibility	Can a fresh run recreate the same artifact from the saved files and rules?
Auditability	Can you inspect the commands, checks, and source trail that produced the result?
Privacy boundary	Which files stay local, and which contents may still traverse the network?
Latency / cost	Is the wall-clock gain worth the token, debugging, and supervision cost?

Why this matters for economists: an impressive demo is not enough. A research workflow has to be checkable and defensible.

What Makes Something “Agentic”? Four Features

- **Tool use** — invokes pandas, curl, R, statsmodels, the shell, whatever the task requires
- **Autonomy** — plans a multi-step empirical workflow without re-prompting.
e.g., “download FRED → clean → regress → plot → save table” as one command
- **Iteration** — observes that a regression failed, identifies the bad column, fixes the formula, reruns
- **Persistence** — remembers your project conventions across sessions (variable names, preferred packages, validated benchmarks)

All four are cumulative. Tool use without autonomy = a glorified Python REPL. Autonomy without iteration = a script that dies on the first error. You need all four for “agentic.”

Spectrum: Chat, IDE Assistant, Terminal Agent

	Web chat	IDE assistant	Terminal agent
Best at	thinking, drafting	editing local code	end-to-end execution
Context	conversation + uploads	open files	full project tree
Typical role	architect's whiteboard	coding copilot	workflow operator
Economic example	design the specification	clean one regression file	run the full pipeline

Concrete example today: Claude Code. **General lesson:** the same workflow logic also appears in Codex CLI, Gemini CLI, Aider, and similar terminal-agent tools.

The Bridge: How a Chatbox Becomes a Terminal Agent

One key technical change: modern LLMs (post-2023) support **structured tool calls**. Instead of writing “you should run `python solve.py`,” the model emits a machine-readable request:

```
{ "tool": "Bash", "command": "python aiyagari_v0.py" }
```

A **harness**—software installed on your Mac—executes this and returns the real output. The model reads it and decides what to do next. This loop is what makes something “agentic.”

	claude.ai (browser)	Claude Code (terminal)
LLM model	same (e.g. Opus 4.5, Nov 2025)	
Harness	none (sandboxed)	local CLI on your Mac
Can run Python?	no	yes— <i>your</i> Python, <i>your</i> machine
Sees real errors?	no	yes—loops back automatically

Same brain, different arms. The terminal agent is not a magically smarter model. The harness is what flips the switch from text-only output to text + actions inside a project.

Early Demo: From Prompt to Research Artifact

Running example: a MEPS pilot asking: *“Build a 2012–2014 uninsurance panel by age group with survey weights.”*

1. **Prompt:** define the question, weights, and desired output
2. **Plan:** list files to download, harmonize, and validate
3. **Run:** parse the raw files, map year-specific variables, compute weighted rates

4. **Validate:** compare 2013 to an external benchmark, inspect the weight variable, save the crosswalk
5. **Output:** one figure, one CSV, one short note saying what passed and what remains uncertain

Same loop, different artifact:

- paper review → structured critique
- fellows dataset → provenance-aware CSV
- Aiyagari model → benchmarked solver output

Reason for showing this early: the workflow should be concrete before we start naming the project objects that support it.

Setup

The durable parts of setup — Git, project folders, and audit trail

What Belongs in the Main Deck vs. the Appendix

Two kinds of setup advice age very differently.

- **Dated and tool-specific** — install commands, package names, subscription tiers, model IDs, pricing. These change every few months and belong in the appendix at the end of the lecture (T6).
- **Durable and concept-level** — version control, project folder layout, file-driven workflow, audit trail. These survive every tool change and belong here in the main deck.

Practical rule: if a slide would need rewriting when you switch from Claude Code to Codex CLI to Gemini CLI, it is operational, not conceptual.

Why Git, Not OneDrive or Dropbox

The risk without Git:

- Claude Code can edit dozens of files autonomously in one session
- OneDrive and Dropbox sync by *copying*—two simultaneous writes create corrupt merge files
- No undo: if Claude makes bad changes, there is no “revert” without Git
- No audit trail: you cannot see what changed or when

What Git gives you:

- `git diff` shows exactly what the agent changed before you accept it
- `git checkout -- .` reverts all uncommitted changes instantly—your emergency brake
- `git log` shows every change with a timestamp and message
- Works offline; cloud sync races with the agent's rapid file writes

Practical rule: store large data files on OneDrive. Store code and scripts in Git. Never use the same tool for both.

First Git Setup: Three Minutes

One-time global setup:

```
git config --global user.name "Your Name"
git config --global user.email "you@email.com"
"
```

Per-project setup:

```
cd my-project
git init
echo "data/ output/ *.pdf" > .gitignore
git add .gitignore
git commit -m "initial setup"
```

You do not need to know Git deeply. Three commands—`git add -A`, `git commit -m "..."`, and a non-destructive restore workflow agreed with your instructor—are enough to work safely with a terminal agent.

The one rule:

- Commit *before* every agent session
- Commit *after* validating results
- That is enough to start safely

Emergency brake:

```
git checkout -- .
```

Undoes *all* uncommitted file changes instantly.
Your safety net when the agent goes wrong.

Looking Ahead: T2 Opens the Hood

T1 framed the problem: chat alone is not enough; a harness wraps tools, files, and verifiers around the model so the loop can do real work.

- T1 told you *what* agents do and *why* the harness exists
- T2 shows you *how* the harness works: components, file hierarchy, initialization, permissions, and the recommended first-session flow
- Then T3 introduces the homotopy workflow that gives the harness something disciplined to do

Arc: T1 Framing → T2 Under the Hood → T3 Homotopy → T4 Project Org → T5 Mistakes/Context → T6 Wrap.

AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 9.1b: The Agentic-AI Landscape (mid-2026)

Zhigang Feng

Spring 2026

Topic 9.1b Agenda

T1 framed the terminal-agent workflow. This interlude widens the lens to the broader agentic-AI landscape an economist now operates inside.

1. **The interoperability stack** — MCP, A2A, AGNTCY/ACP, neutral governance
2. **Orchestration patterns** — frameworks, context engineering, the multi-agent cost-benefit
3. **Reasoning models & inference-time compute** — think budgets, measured time horizons
4. **Autonomous coding & computer-use agents** — scheduled runtimes, sandboxing
5. **Benchmarks & measurement validity** — ties to the course's measurement theme
6. **Safety & reproducibility** — untrusted inputs, chain-of-custody

Reading convention: [solid] = primary or citable; [hype] = vendor self-report (counter-source given).

Facts current as of June 2026. Model versions are stated with release dates; the field moves monthly — recheck before teaching.

The Interoperability Stack

Compete on models, cooperate on plumbing

Three Layers of Agent Interoperability

The 2025–26 story is standard-setting: vendors compete on models but increasingly cooperate on the wiring between agents and tools.

Layer	Protocol	What it standardizes
Agent ↔ tool	MCP (Model Context Protocol)	how an agent discovers and calls external tools/data sources
Agent ↔ agent	A2A (Agent-to-Agent)	how independent agents delegate tasks to one another
Discovery / identity	AGNTCY / ACP	agent directories, capability cards, identity and trust

- MCP was introduced by Anthropic (Nov 2024); it is now *multi-vendor*, not proprietary.
- **OpenAI adopted MCP** at DevDay (Oct 6, 2025); Google and Microsoft tooling support it too. **[solid]**
- Governance moved to a neutral home: the **Linux Foundation Agentic AI Foundation (AAIF)**, formed Dec 9, 2025, now hosts MCP and related projects. **[solid]**

Why Neutral Governance Is a Network-Effects Story

For economists, the interoperability stack is a textbook standard-setting / network-effects case.

- A shared protocol lowers switching costs: a tool wrapped once in MCP works across Claude, ChatGPT, Gemini, and open agents
- Network effects make the standard more valuable as adoption grows — which is why a single-vendor protocol faces adoption resistance
- Moving MCP under the **Linux Foundation AAIF** (Dec 9, 2025) is the classic move: hand the “plumbing” to a neutral body so rivals will build on it
- Competition then concentrates where rents remain: model quality, latency, and price — not the connectors

Reproducibility note. Protocols version. Pin the *spec version* you used (e.g. MCP spec date, A2A version) alongside the model version and date — a tool call that worked under one spec revision may behave differently under the next.

Orchestration Patterns

How multi-step agent systems are actually built

Four Orchestration Patterns

Production agent systems are built from a small set of recurring control structures.

Pattern	Shape
Orchestrator–worker	a lead agent decomposes a task and dispatches workers; <i>dominant in production</i>
Planner–executor	one component plans, another executes step by step
Reflection	the agent critiques and revises its own output before returning
Handoff	control passes between specialized agents (e.g. triage → specialist)

The terminal-agent workflow you learned in T1–T5 is an **orchestrator–worker system**: the main agent orchestrates; code-reviewer, domain-reviewer, and verifier sub-agents are the workers. The landscape just adds vocabulary and tooling around the same loop.

Frameworks Reached 1.0 (2025–26)

The orchestration layer matured from research demos to versioned, supported frameworks.

Framework	Milestone	Vendor
LangGraph 1.0	Oct 2025	LangChain
OpenAI Agents SDK (+ AgentKit)	SDK Mar 2025	OpenAI
Google Agent Dev. Kit (ADK)	v1.0 May 2025	Google
Microsoft Agent Framework	1.0 Apr 2026	Microsoft
CrewAI	1.0	CrewAI
Anthropic Claude Agent SDK	Sep 2025	Anthropic

Naming note. What was the *Claude Code SDK* was renamed the **Claude Agent SDK** on Sep 29, 2025, reflecting generalization beyond coding. **[solid]**

Durable lesson: the patterns outlive the framework. Learn orchestrator–worker once; map it onto whichever SDK your institution adopts.

The Multi-Agent Cost–Benefit Decision

More agents is not free. Treat it as marginal value vs. marginal cost.

- Anthropic reports its multi-agent research system beat a single-agent baseline by **+90.2%** on its internal research-eval. . .
- . . . at roughly **15× the token cost**
- So multi-agent pays off when the task is broad, parallelizable, and the answer's value is high relative to tokens
- It is wasteful for narrow, sequential, or cheap tasks

[solid, label: **Anthropic-internal-eval**] — a vendor benchmark on a self-defined research task, not an external suite. Read the +90.2% as directional, not portable.

Economist framing. You are choosing a point where marginal research value = marginal token (and latency, and debugging) cost. The Aiyagari sub-agent review in T4 was justified on *wall-clock*, not because “more agents are better.”

Context Engineering Displaced Prompt Engineering

The 2025–26 shift: managing what is in the context window now matters more than wording a clever prompt.

- **Compaction.** Summarize and clear stale turns so the working set stays in the high-recall part of the window (the “lost in the middle” problem from Lec07 T3b).
- **External memory.** Persist state to files (`NOTES.md`, `progress.md`, `CLAUDE.md`) rather than relying on the chat history.
- **Sub-agents with clean context windows.** Hand a worker a fresh, focused context instead of the orchestrator’s accumulated noise.

Why this matters for research: it is exactly the discipline T4–T5 already taught — file-driven workflow, progress logs, focused reviewers. The landscape gave it a name; the habit was already correct.

Reasoning Models

Tool use in the reasoning loop, and how long agents actually run

Reasoning Models and Think-Budget Control

Frontier models now spend variable inference-time compute, and increasingly call tools inside the reasoning loop.

- **Tool use in the loop.** OpenAI o3 / o4-mini (Apr 2025) interleave tool calls with chain-of-thought rather than reasoning first and acting after.
- **Think-budget routers.** GPT-5 (Aug 2025) → GPT-5.5 (Apr 2026) route between fast and deep modes per query.
- **Toggleable extended thinking + configurable token budget.** **Claude Opus 4.5** (Nov 2025) lets you turn extended thinking on/off and cap the thinking-token budget; it was the first model past **80% on SWE-bench Verified**, and Opus pricing was cut to \$5/\$25 per Mtok (input/output).
- **Parallel deep reasoning.** Gemini “Deep Think” explores multiple reasoning paths in parallel.

Benchmark caveat (carry this everywhere). The 80% SWE-bench Verified figure is real but the metric is contaminated/saturating: OpenAI *stopped reporting* SWE-bench Verified in Feb 2026. Pair any Verified headline with SWE-bench Pro (~55–59% at the frontier, held-out). **[solid]**

How Long Can an Agent Actually Work? Measure, Don't Trust Marketing

Vendors say “works for hours” or “runs for days.” METR gives a measured alternative.

- METR’s *task-length-doubling* result: the task length an agent completes at **50% reliability** has doubled roughly every **7 months**, accelerating to **~4 months** in 2024–25. **[solid]**
- Time Horizon 1.1 (Jan 2026) pegs **Opus 4.5 $\approx$ 320 min** and **GPT-5 $\approx$ 214 min** (50%-reliability horizon).
- Use these measured minutes instead of vendor “multi-hour/multi-day autonomy.”

Metric fragility. The number is tied to a **50% reliability** threshold and a specific task suite; the horizon at 80% reliability is much shorter, and it shifts across suites. A measured-but-fragile number still beats a vendor claim.

[hype counterpoint] product pages advertising “multi-hour” or “multi-day” autonomy — no reliability threshold stated.

Research translation: the right question is not “can it run for hours?” but “at what reliability, on what task distribution, validated how?”

Autonomous Coding & Computer Use

From interactive assistants to scheduled, sandboxed runtimes

Coding Agents Became Scheduled, Autonomous Runtimes

In 2025–26, coding agents shifted from interactive assistants to background runtimes with persistent state.

- **OpenAI Codex “Goal Mode”** became the default (May 2026), with persistent cross-session state — the agent resumes a goal rather than starting fresh each chat.
- **Anthropic “Code with Claude” (May 6, 2026)** reframed Claude Code as an *orchestrator*:
 - **Auto Mode** with prompt-injection / destructive-action classifiers + human approval gates
 - **Routines** triggered by cron or webhook
 - **Worktrees** for parallel branches
 - sandboxed **Managed Agents** with scoped credentials

Lesson for research infra. Before an agent runs unattended on real data or repos, configure sandboxing, credential scoping, and approval gates first. “Autonomous” is a permissions decision, not a model capability.

Computer-Use Agents: Strong Benchmarks, Volatile Products

Browser/computer-use agents got good on paper but the standalone products churned.

- Computer-use / browser agents reached **~78–84% on OSWorld-Verified** (human baseline ~72%).
- **But standalone products are volatile:** Google *killed* Project Mariner (May 2026); OpenAI folded Operator into ChatGPT Agent / Atlas.

Flag the benchmark numbers. OSWorld scores are **scaffold-dependent and largely self-reported**: the same model scores very differently under different harnesses, and “above human baseline” does not mean reliable on *your* unseen workflow. Treat as directional. **[solid claim that products churned; benchmark figures = vendor/scaffold-dependent]**

Lesson: do not build durable research infrastructure on a single experimental product. Build on the protocol layer (MCP) and your own files; treat any one browser agent as replaceable.

Benchmarks & Measurement Validity

The course's measurement theme, applied to agents

SWE-bench Verified Collapsed Under Scrutiny

This module is the measurement theme of the course pointed at agent evals: a headline number is not a measurement until you know its validity.

- Contamination + harness-hacking critiques eroded SWE-bench Verified; OpenAI stopped reporting it in Feb 2026.
- Berkeley RDI *trivially broke four agent benchmarks via the harness* (Apr 2026) — exploiting the scaffold, not solving the task.
- Always pair a Verified headline with **Scale's SWE-bench Pro** (~55–59% frontier, held-out problems).
- **Reliability under repetition** matters more than best-of- n for autonomous use: τ -bench *pass-hat- k* falls from ~90% single-try to ~57% at $k=8$.

Hard rule. Never put a SWE-bench *Verified* score and a SWE-bench *Pro* score on the same axis — different problem sets, different difficulty, different contamination exposure. **[solid]**

Single-Try Score vs. Reliability Under Repetition

For an agent that runs unattended, the relevant statistic is not its best run — it is whether it succeeds every time.

Best-of- n thinking (wrong for autonomy)

- report the best of several attempts
- flatters the model
- fine for a human-in-the-loop draft you will check

pass-hat- k / reliability (right for autonomy)

- probability *all* k independent runs succeed
- τ -bench: $\sim 90\%$ at $k=1 \rightarrow \sim 57\%$ at $k=8$
- this is what matters when no human checks each run

Economist translation: an unattended agent on a recurring data pull is a repeated trial. A 90% single-try success compounds into frequent failure across a quarter of nightly runs. Design for the reliability curve, not the demo.

Safety & Reproducibility

Untrusted inputs and chain-of-custody for agentic research

Treat Everything the Agent Reads as Untrusted

The defining agentic-AI security failure is prompt injection: data the agent merely reads can hijack what it does.

- **OWASP Top-10 for Agentic Applications** (Dec 2025) now codifies these risks.
- Documented exploits: **EchoLeak** (CVE-2025-32711, zero-click M365 Copilot data exfiltration); **ShadowPrompt** in Claude's Chrome extension; Anthropic's **GTG-1002** — the first largely-autonomous cyberattack (Nov 2025).
- **NIST CAISI** red-teaming raised task-hijacking success from **11% to 81%** with adaptive attacks.

Operating rule. Treat any content the agent reads — emails, web pages, retrieved documents, even tool descriptions — as an *untrusted injection channel*. Use least-privilege sandboxing and revocable permissions, exactly as T4's `settings.json` deny list and T5's security frame prescribed. **[solid]**

Reproducibility: Chain-of-Custody for Agentic Results

Can an agent reproduce a published computational result? Often only partly — which makes logging non-negotiable.

- **CORE-Bench:** agents reproduce published computational results only **~60% (easy)** / **~21% (hard)**. **[solid]**
- Therefore **log full trajectories:** prompts, tool calls, seeds, *model versions*, *spec versions*, and *dates* — the chain-of-custody for any agent-produced artifact.

Read vendor safety numbers skeptically. Distinguish a vendor's “~1% attack success on a *fixed* suite” from academic findings that *adaptive* attacks still exceed ~85%. A fixed-suite number measures the suite, not the threat model. **[solid]**

Ties to T6. This is the same six-criteria discipline (reproducibility, verifiability, auditability) — now with the dates and spec versions the landscape forces you to pin.

Synthesis: The Landscape Reinforces the Workflow Spine

Every module above maps back onto the terminal-agent workflow of T1–T6 and onto the course's measurement / identification theme.

Landscape module	Connection to the spine
Interoperability (MCP/A2A)	build on the neutral protocol layer, not one product; pin spec versions
Orchestration	T4's sub-agent review <i>is</i> orchestrator–worker; pay for multi-agent only when value > token cost
Reasoning & time horizons	“runs for hours” → a measured, reliability-thresholded number
Coding / computer-use	autonomy is a sandboxing + approval-gate decision, set before the run
Benchmarks	a headline score is not a measurement until you know its validity (Verified vs. Pro)
Safety & reproducibility	untrusted-input discipline + logged chain-of-custody (dates, versions, seeds)

The plumbing changes monthly; the research discipline does not.

AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 9.2: Under the Hood, First Session, and the 5-Pillar Workflow

Zhigang Feng

Spring 2026

Topic 9.2 Agenda

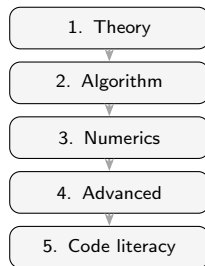
1. **The 5-Pillar Workflow** — recap from Lec06
2. **Under the Hood** — harness, LLM, file system, initialization
3. **Your First Session** — a concrete walkthrough
4. **The 5-Pillar Workflow Detailed**

Arc: T1 Framing → T2 Under the Hood → T3 Homotopy → T4 Project Org → T5 Mistakes/Context → T6 Wrap.

The 5-Pillar Workflow (Recap from Lec06)

Every agent project in this lecture and Lec10 traces five pillars. The agent accelerates each pillar; it does not replace any of them.

1. **Theory** — what you are trying to model or measure
2. **Algorithm** — the math you are going to compute
3. **Numerics** — discretization, solver choice, tolerance
4. **Advanced** — extensions, robustness, alternative specifications
5. **Code literacy** — reading and reasoning about the code, not just typing prompts



We will return to these pillars in the workflow-detail section of this deck and again in T6 evaluation.

Under the Hood

Harness, memory, permissions, and the control loop

Terminal Vocabulary for Economists

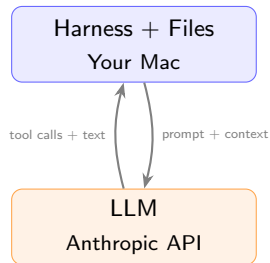
Term	Meaning
Terminal	The text window where you type commands instead of clicking buttons.
Shell	The program that interprets commands. macOS often uses <code>zsh</code> ; many examples are written in <code>bash</code> .
Bash	A common shell language. When Claude Code says it will use Bash, it means “run this command in the terminal.”
Working directory	The folder commands run from. <code>cd my-project</code> changes it; <code>pwd</code> prints it.
PATH	The list of folders your shell searches when you type a command such as <code>python</code> or <code>claude</code> .

Why this matters: terminal agents act through shell commands. If the working directory or PATH is wrong, the agent may be competent but operating in the wrong place.

Three Components of a Terminal-Agent Setup

A terminal agent (Claude Code, Codex CLI, Gemini CLI, Aider) has three physically distinct parts:

1. **The Harness** (lives on your Mac)
CLI software you install. Reads your files, runs shell commands, manages the conversation loop.
2. **The LLM** (lives on the provider's servers)
Every message is bundled with conversation history and sent via HTTPS to the API. The “thinking” happens in the cloud.
3. **Your Files** (live on your Mac)
Python scripts, data CSVs, LaTeX—all on your disk. Code executes on your hardware with your packages.



Key: code can run locally and many project files stay on your machine. That does *not* mean the whole workflow is automatically private.

Privacy Boundary: Local Execution Is Not the Same as Local Processing

- code may execute locally on your machine
- many project files may stay on disk locally
- if file contents, prompts, or tool outputs are sent back in the conversation, those contents may still traverse the network
- only a fully local or institutionally private deployment changes that boundary

Practical rule: local execution is not a license to paste restricted data into an agent workflow.

The File Hierarchy: Global vs. Local

Two layers of configuration. **Local overrides global.**

Path (Claude Code example)	Scope	What lives here
~/.claude/settings.json	Global	Default permissions, model preference
~/.claude/MEMORY.md	Global	Cross-project memory (persists forever)
~/.claude/skills/	Global	Skills available in every project
<project>/CLAUDE.md	Local	Project brain, read every session
<project>/.claude/settings.json	Local	Project permissions (overrides global)
<project>/.claude/skills/	Local	This project's slash commands
<project>/.claude/agents/	Local	Specialized reviewer roles
<project>/.claude/rules/	Local	Always-on guardrails

Other agents follow analogous layouts. Codex CLI uses AGENTS.md; Gemini CLI uses GEMINI.md. The pattern — global defaults plus a project-specific system prompt — is durable across products.

Rule: global dotfolder = your personal defaults (apply everywhere). Local dotfolder = the project's rules (apply only here).

Initialization: What Happens When You Type `claude`

Five steps run **before you type your first message**:

1. **Read global config** — loads `~/.claude/settings.json`: permission defaults, model preference
2. **Detect project context** — searches current directory (and parents) for `CLAUDE.md` and `.claude/`
3. **Load project brain** — reads `CLAUDE.md` into the context window
4. **Auto-load all rules** — every `.claude/rules/*.md` file is loaded silently
5. **Ready** — context window pre-populated before you say anything

Analogy: step 4 is a professor re-reading lecture notes before class. Students arrive; context is already loaded.

Cost warning: steps 3–4 consume tokens before you type anything. Keep `CLAUDE.md` and rules files concise.

Tokens and Cost: Putting Cost on the Radar

Every agent turn is a model API call. Cost scales with input and output tokens.

- **Input tokens.** Your prompt, conversation history, CLAUDE.md, rules, retrieved chunks. Every step that adds context adds cost.
- **Output tokens.** Generated code, JSON tool calls, prose explanations. Reasoning-heavy modes also produce many internal tokens.
- **Long sessions accumulate context.** Use context-window compaction commands (/compact, "summarize and clear") to free the window; use cost trackers (/cost) to see USD spent so far.
- **Pin for reproducibility.** Record the model version, prompt template, and seed when applicable; AEA disclosure (Oct 2024) extends to AI-assisted analysis.

Lec07 T3a contains the pricing table and reproducibility detail. The point here is that cost is a parameter of the workflow, not a footnote.

The Permission System and Local Code Execution

Claude's toolbox:

Tool	What it does
Bash	Run any shell command
Read	Read a file from disk
Write	Create or overwrite a file
Grep/Glob	Search content / find files
Agent	Spawn a child Claude instance

The permission model:

- **Default:** Claude asks before every Bash call. You see the exact command first.
- **Whitelist:** add trusted commands to `.claude/settings.json`:

```
"allowedTools": [  
  "Bash(python *)",  
  "Bash(git *)" ]
```

- **Bypass-permissions:** skips all prompts. Use only when you fully trust the task.

“Which Python ran this?” (common confusion):

- Claude's Bash inherits your shell's `$PATH`
- If you ran `conda activate myenv` before claude, *that* Python runs
- Add a `which python` check to your `CLAUDE.md` to confirm

Key Modes: Plan, Effort, and Session Control

Command / Key	What it does	When to use
Shift+Tab	Plan mode: Claude writes a plan, does NOT execute. Waits for approval.	Start every non-trivial task here
/effort max	Maximum reasoning depth: best quality, slower, costlier	Novel problems, economic correctness
/effort default	Balanced quality and speed	Routine work
/cost	Show tokens used + estimated USD for this session	Budget check before long tasks
Esc	Interrupt Claude mid-task immediately	When it's going the wrong way
/compact	Summarize conversation to free context window	After ~20 turns; between phases
/clear	Wipe context, start fresh	New unrelated task

The effort–cost tradeoff:

- Maximum-effort modes plus long context can be an order of magnitude more expensive than default. See Lec07 T3a for current pricing.
- Spend effort on *decisions*; save it on *execution*

Your First Session

The recommended workflow from scratch

Your First Five Minutes: A Concrete Walkthrough

T2 has so far explained the initialization in the abstract. Here is what it actually looks like.

```
$ cd ~/research/aiyagari-pilot
$ claude
[reads ~/.claude/settings.json]
[reads ./CLAUDE.md]
[loads .claude/rules/iterative-workflow.md]
[ready]

You> Review aiyagari_v0.py. Is the budget constraint correctly specified?

claude> [Read aiyagari_v0.py]
        [Read .claude/skills/solve-model/references/aiyagari-guide.md]
        The constraint  $c + a' = (1+r)a + w \cdot l$  checks out, but I notice ...
```

1. Harness initialization is automatic; no flags needed for the first session.
2. The agent reads context (rules, reference guides) *before* answering — visible as bracketed [Read ...] traces.
3. Each [Read ...] is a JSON tool call (T1) that the harness translated into a real disk read.
4. Your message can be a question, a command, or a slash command — the response loop is the same.

Start Here: The Recommended Workflow for New Users

0. Before Claude—pen + paper

Write one sentence: *"I want to compute X using method Y for question Z."*

1. Enter plan mode (Shift+Tab)

Describe your goal. Claude drafts a plan. *Nothing runs yet.*

2. Review and approve

Read the plan. Ask questions. Revise. Type "approved."

Why this matters: this is the cheapest place to catch a wrong task definition.

Common beginner mistake: skipping Step 0. Without a clear goal, Claude will confidently build the wrong thing.

Continue: Steps 3–5

3. Implement

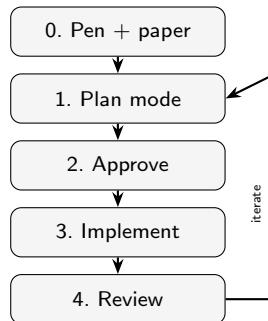
Claude executes the approved plan. Press Esc to stop if wrong.

4. Review output

Does it make economic sense? Re-derive one number by hand.

5. Iterate

Fix, extend one feature, compact context, and return to step 1.



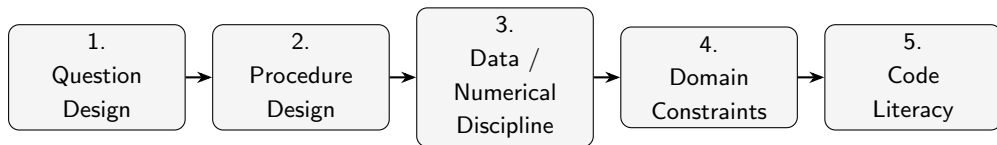
Practical takeaway: after each implementation pass, force one human check before you ask for the next extension.

The 5-Pillar Workflow Detailed

The research-workflow rendering of the Lec06 pillars

The Research-Workflow Rendering of the Five Pillars

The Lec06 pillars (theory, algorithm, numerics, advanced, code literacy) reappear here as the agent-assisted research workflow: question design, procedure design, data/numerical discipline, domain constraints, code literacy.



The throughline of the lecture:

- Pillars 1–4 determine what should be built and what counts as success
- Pillar 5 lets you verify what the agent actually built
- If the pillars are weak, agent output becomes faster nonsense

Pillar 1: Question Design

Before any workflow, decide what the economist is actually trying to learn.

- State the question in economist language: model object, empirical estimand, review brief, or dataset goal
- Decide what constitutes evidence: policy function, coefficient, figure, quote-level comment, or provenance-aware record
- Write down limiting cases before any code exists

Cross-domain point: if the question is vague, the workflow can still look polished while answering the wrong thing.

Pillar 2: Procedure Design

After the question is clear, decide what procedure is actually appropriate.

- Choose the right workflow: Bellman iteration, MEPS harmonization pipeline, paper-review decomposition, or batched metadata collection
- Decide what the acceptance criterion is: convergence, benchmark recovery, quote verification, or source completeness
- Convert the question into a bounded procedure the agent can implement and you can inspect

Cross-domain point: the agent can write VFI code, merge MEPS files, or draft a review report, but only you know which workflow is appropriate for the task.

Pillars 3 and 4: Data/Numerical Discipline and Domain Constraints

Pillar 3: Data / numerical discipline

- Grid design, interpolation, shock discretization, and stability checks for models
- Schema harmonization, weights, crosswalks, missingness, and saved source logs for data work
- Recognition of failure modes: kinks, dropped weights, broken mappings, or silently weak sourcing

Pillar 4: Domain constraints

- Advanced methods still matter: projection, perturbation, clustering decisions, source restrictions, claims discipline
- Knowing when the agent's convenient default is computationally, econometrically, or substantively wrong

Economics point: a workflow can run cleanly and still be wrong. Agents reduce coding cost; they do not remove approximation error, data fragility, or source discipline.

Pillar 5: Code Literacy and Division of Labor

You provide	Agent provides
Question, identification logic, and source rules	Fast code generation and refactoring
Workflow choice and benchmark design	Boilerplate implementation and debugging
Knowledge of the data, model, and institutions	File edits, script writing, and test runs
Economic interpretation and seminar defense	Iteration on visible errors and outputs

Research version of the slogan: the agent can be a strong RA, but you still have to be the principal investigator.

Validation Is Your Referee Checklist

Every iteration needs both workflow checks and economic checks.

Workflow checks

- Convergence achieved
- No NaN or Inf values
- Mappings, weights, or source logs saved where expected
- Diagnostics, residuals, or quote checks pass
- Script reproduces the same output twice

Economic checks

- Limiting cases recover known results
- Comparative statics, subgroup patterns, or review claims make sense
- Magnitudes and source coverage are plausible
- Inference uses the right standard errors, sample, and restrictions
- You can explain the result in plain economic language

If you cannot defend the output in a seminar, you are not done.

Looking Ahead: T3 Gives the Workflow a Spine

T2 showed you the harness, the initialization, and the recommended first-session flow. T3 turns that into a disciplined research workflow.

- T2 covered the *mechanism*: what runs, where files live, how the loop is initialized and constrained
- T3 introduces the *homotopy workflow*: build the easy version, validate, then extend one margin at a time
- By the end of T3 you will have run the OLG five-step demo as a concrete instance

*Arc: T1 Framing → **T2 Under the Hood** → T3 Homotopy → T4 Project Org → T5 Mistakes/Context → T6 Wrap.*

AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 9.3: The Homotopy Workflow and Tool Choice

Zhigang Feng

Spring 2026

Topic 9.3 Agenda

1. **The Homotopy Workflow** — iterative refinement
2. **OLG Five-Step Demo** — a worked instance
3. **Tool Choice** — selecting tools and sub-agents

*Arc: T1 Framing → T2 Under the Hood → **T3 Homotopy** → T4 Project Org → T5 Mistakes/Context → T6 Wrap.*

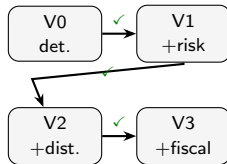
The Homotopy Workflow

Build from simple to complex, validate, then extend

The Homotopy Workflow

In workflow design, homotopy means solving an easier version first and then extending it toward the full research task.

- Start from a version with clear intuition and known checks
- Use that validated baseline as the starting point for the next
- Attribute new bugs to the new margin you just added
- Avoid asking the agent to solve the full complicated problem in one jump



Each step small; each step validated; if a step fails, return to the last validated version.

Four examples of the same logic:

deterministic model → 3-year MEPS pilot → one-section review → seed roster only

Why this matters for agentic coding: the agent performs best when each step is bounded, testable, and attached to an existing working baseline.

Homotopy in Practice: Build from Simple to Complex



This replaces the fantasy prompt “solve my project.”

- Start with a problem you can state and benchmark
- Give the agent bounded tasks with visible acceptance criteria
- Add one new ingredient at a time

Step 1: Design the Economic Problem

Before any code, write down the task in economist language.

- Research question in one sentence
- Objective and constraints, or the relevant source rules
- Equilibrium definition, estimand, or review standard if applicable
- Parameters, sample, source universe, and economic motivation
- Outputs you will treat as evidence

Aiyagari anchor:

$$\max \mathbb{E}_0 \sum_{t=0}^{\infty} \beta^t u(c_t) \quad \text{s.t.} \quad c_t + a_{t+1} = (1+r)a_t + wz_t, \quad a_{t+1} \geq \underline{a}$$

Same principle elsewhere: a vague MEPS brief, a vague review request, or a vague fellows-data assignment also produces output that may be internally clean but substantively wrong.

Step 2: Write the Specification File

The specification file is the contract between you and the agent.

A good specification file states four things clearly:

1. assumptions and domain rules
2. data or source restrictions
3. procedure and validation target
4. required outputs and where to save them

Examples: CRRA utility and convergence tolerance for Aiyagari; weights and subgroup cuts for MEPS; quote-level evidence rules for paper review; official/institutional-source rules and provenance fields for fellows data.

Best practice: ask the agent to restate the specification in its own words before it writes code. That catches misunderstandings when they are still cheap.

Step 3: Choose the Procedure Explicitly

Pseudocode or a phase plan is where you catch design mistakes before implementation.

```
Phase A: ingest or define the object  
Phase B: run the chosen procedure  
Phase C: save diagnostics and checks  
Phase D: report only what passed validation
```

Concrete procedures: VFI or EGM for Aiyagari, a 3-prompt pipeline for MEPS, section decomposition for paper review, and batched metadata collection for fellows data.

The point is not elegance. The point is that you and the agent agree on the procedure before code details begin to hide conceptual errors.

Step 4: Build V0 First

V0 is the simplest version with a benchmark you understand.

Aiyagari V0: deterministic income

- If $\beta(1+r) = 1$, savings policy should sit on the 45-degree line
- If $\beta(1+r) < 1$, the agent decumulates toward the borrowing constraint
- If $\beta(1+r) > 1$, assets grow

Discipline: do not ask the agent to add income risk, stationary distributions, and equilibrium search before V0 already passes the simple checks.

What “V0 First” Means Across Four Tasks

Task	V0	First validation target
Aiyagari model	deterministic household problem	recover simple savings logic and boundary behavior
MEPS pipeline	3-year pilot panel	weighted rates line up with an external benchmark
Paper review skill	one section or theorem block	quotes are real and comments are specific
Fellows dataset	seed roster only	source rules and provenance fields are actually recorded

Why economists should care: V0 keeps the first error interpretable. You want the first failure to be obvious, not hidden inside a giant workflow.

Steps 5 and 6: Validate, Then Extend One Margin at a Time

Version	New ingredient	Required benchmark
V0	Deterministic household	analytical cases
V1	One new model, data, or review margin	setting the new margin to zero recovers V0
V2	Saved diagnostics and state tracking	mass sums, benchmarks, quote checks, or source logs behave sensibly
V3	Full target workflow	final artifact survives validation and interpretation

Why this works with agents: when a new version breaks, both you and the agent know where to look.

Why This Workflow Works for Economists

- It keeps theory, computation, and validation aligned
- It gives the agent bounded tasks instead of open-ended wishes
- It preserves an audit trail of how the baseline was constructed
- It mirrors how good research assistants are actually trained: start with a simple task, verify it, then widen scope

Main lesson: the workflow is the real product. The particular agent you use can change.

Why “Homotopy Workflow”? Borrowing from Numerical Methods

From numerical methods: homotopy continuation starts from a problem with a known solution and *continuously deforms* toward the target. The same principle works for AI-assisted coding (see Judd 1998, *Numerical Methods in Economics*, Ch. 5–7 for the perturb-and-sustain approach in macro).

- V0 is the “easy problem” with a known solution
- Each version continuously deforms toward the research target
- Each validated version is the *foundation* for the next
- If V2 breaks, you know the bug is in V2’s new code — not V0 or V1

Why the name matters: the *homotopy workflow* traces a smooth path from a *specifiable* model to a *validated* codebase. $V0 \rightarrow V1 \rightarrow V2$ are not arbitrary milestones — they are intermediate gates along a continuous deformation.

The Homotopy Workflow vs. One-Shot Generation

Four reasons the homotopy workflow is especially effective with agents:

1. **Agents excel at well-specified, bounded tasks.**

“Solve my model” fails. “Implement V0 per this pseudocode and validate against these 3 benchmarks” succeeds.

2. **Starting simple gives both you and the agent a working codebase.**

V1 inherits validated V0 code. The agent builds on tested functions, not a blank file.

3. **Bug attribution is immediate.**

If V2 breaks, the bug is in V2’s new lines, not the validated inherited ones.

4. **Context builds naturally.**

By V3, the agent has seen the spec, pseudocode, three validated versions, and every correction.

Deep context — not a 10-page prompt.

Discipline: resist the urge to skip ahead. A working V0 takes 30 minutes. Debugging a broken V2 because V0 was never validated takes a full day.

OLG Five-Step Demo

A concrete homotopy workflow for computational macro

Demo: Folder Structure Is the Method

Folder or file	Teaching role
stages/	five durable stages: model, equilibrium, algorithm, pseudocode, implementation/validation
prompts/v*_to_v*.md	one bounded terminal-agent session per version transition
versions/v0-v5	validated working code preserved at every step
tests/	one smoke test per version; failures identify the new margin
build/*.json	reproducibility reports from terminal runs

Demo path: demos/M4_olg_5step_demo/ (local copy in this lecture; mirror of MiniCourse_8hr M4_olg_5step_demo minus .venv/build/).

Version Ladder: One Economic Margin at a Time

Ver.	Adds	Validation gate
V0	3 cohorts, 1 asset, 2-state TFP, one NN policy	deterministic steady-state collapse; procyclical capital; RMS Euler residual below threshold
V1	7 cohorts and lifecycle labor	V0 logic preserved; middle-aged consumption peak appears
V2	4-state Markov TFP	collapse back to two states reproduces V1
V3	bonds, price, clearing, complementarity	zero bond-loss weight reproduces V2; positive late-life holdings
V4	capital adjustment cost	zero adjustment cost reproduces V3; capital path smooths
V5	stabilizing homotopy phases	residuals fall by phase; final Euler error target met

Research lesson: the agent never receives "solve the big model." It receives one validated deformation at a time.

Demo Launch and Classroom Contract

```
cd Lec09_Agentic_AI/demos/M4_olg_5step_demo
pip install -e .
python3 -m unittest discover -s tests
python3 run_demo.py
jupyter notebook demo.ipynb
```

- **What the demo proves:** homotopy preserves a working baseline while complexity rises.
- **What to inspect:** prompts, version folders, test names, validation messages, and regenerated reports.
- **Failure recovery:** if V3 breaks, return to validated V2 and add only the bond-market-clearing layer.

Tool Choice

Which tool fits which research task

Choose the Tool That Matches the Task

Tool type	Use it for	Economic example
Web chat	framing and explanation	clarify identification logic, draft a model summary
IDE assistant	local code editing	vectorize one function or explain one traceback
Terminal agent	multi-step execution	restructure a project, run scripts, generate diagnostics, and iterate

As of April 2026: Claude Code, Codex CLI, Gemini CLI, and Aider are examples of terminal-agent workflows.

Why the Terminal Agent Often Becomes the Workhorse

- It can operate on the real project rather than on copied fragments
- It can run the code, inspect errors, and update multiple files
- It works well when the task is bounded but longer than one edit
- It still leaves the economist in the approval and validation loop

For empirical research, this is usually the sweet spot: enough autonomy to save time, not so much autonomy that you stop checking the economics.

Looking Ahead: T4 Organizes the Project Around the Workflow

T3 gave you the workflow logic — V0 first, validate, extend one margin at a time. T4 turns that logic into files on disk: skills, agents, rules, hooks, settings.

- T3 covered the *discipline*: how to stage and validate the work
- T4 covers the *infrastructure*: how to encode the discipline in reusable project objects
- Then T5 names the failure modes the infrastructure is supposed to prevent

Arc: T1 Framing → T2 Under the Hood → **T3 Homotopy** → T4 Project Org → T5 Mistakes/Context → T6 Wrap.

AI for Economic Research: Dynamic Models, Language, and Agents
Lecture 9.4: Project Organization — Specs, Harness, Skills, Sub-Agents

Zhigang Feng

Spring 2026

Topic 9.4 Agenda

1. **Project Organization** — specs, project memory, tests, harness, skills
2. **Sub-Agents and Remote/HPC Workflows** — parallel review, move agent to the compute

*Arc: T1 Framing → T2 Under the Hood → T3 Homotopy → **T4 Project Org** → T5 Mistakes/Context → T6 Wrap.*

Project Organization

How a terminal-agent project stays organized

Why Files on Disk Matter

The harness turns today's chat into tomorrow's reusable workflow.

- Conventions live in files, not in your memory
- Validation steps become reproducible lab procedure
- New sessions can start from the project's actual state
- Coauthors and students can inspect what the workflow is supposed to be

Economist translation: the harness is like turning tacit RA practice into written lab protocol.

Why These Project Objects Exist: Coordination

Object	Research pain point it solves	Running example
CLAUDE.md / project memory	You do not want to re-explain the project every session	keep the MEPS weights rule or Aiyagari benchmark visible
Skill	A repeated workflow should not be improvised every time	paper-review pipeline with the same quality gates
Agent	Some review roles are easier to parallelize and specialize	code reviewer vs. domain reviewer

Reason for this slide: these objects are not product trivia. They are answers to recurring research coordination problems.

Why These Project Objects Exist: Control and Credibility

Object	Research pain point it solves	Running example
Rule	Some instructions should always be on	always save outputs to output/ before claiming success
Verifier	Output should pass named checks before it counts	rerun script, inspect diagnostics, confirm files exist
Provenance log	Source discipline must survive beyond the chat	fellows dataset with field-level URLs and notes

Practical takeaway: workflow objects are worthwhile only if they reduce a real coordination or credibility failure.

Minimal Project Tree

```
1 my-research-project/  
2 +-- CLAUDE.md  
3 +-- .claude/  
4 |   +-- skills/  
5 |     |   +-- paper-review/  
6 |     |     +-- SKILL.md  
7 |     |     +-- references/  
8 |     |       +-- referee-template.md  
9 |   +-- agents/  
10 |     |   +-- code-reviewer.md  
11 |     |   +-- domain-reviewer.md  
12 |     |   +-- verifier.md  
13 |   +-- rules/  
14 |     |   +-- iterative-workflow.md  
15 |   +-- settings.json  
16 +-- scripts/  
17 +-- output/  
18 +-- sources/  
19 +-- quality_reports/
```

This is enough to explain the workflow without burying students in infrastructure. The exact research question can change; the organizational logic stays.

CLAUDE.md: The Team-Facing Project Memory

CLAUDE.md is the main shared instruction file for the project.

```
# CLAUDE.md
Project: MEPS uninsurance pilot for lecture 9
Language: Python
Goal: harmonize 2012--2014 files and produce one validated figure
Conventions: no hardcoded paths; save outputs to output/
Checks: use survey weights; save the year map; report what passed
```

Why students should care: it captures the kind of project context you would otherwise re-explain every session.

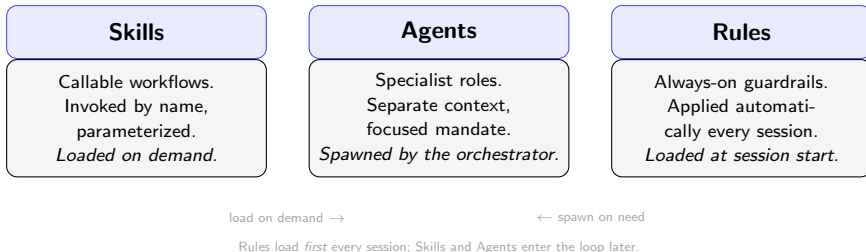
CLAUDE.md, Auto Memory, and Optional Course Logs

Three different persistence ideas should not be confused.

Mechanism	Who writes it	Why it exists
CLAUDE.md	you / team	durable project instructions and conventions
Auto memory	the tool	per-project learned patterns and corrections
Optional course log	you / course	explicit progress notes such as MEMORY.md or progress.md

Course recommendation: rely on CLAUDE.md for shared rules, treat auto memory as helpful but not authoritative, and keep explicit logs when you want students to inspect the evolution of a project.

Skills, Agents, and Rules: Three Different Jobs



Loading order matters. Rules are read before the first user message; Skills load when invoked; Agents launch with the rules and any inherited context as their system prompt.

Plain English vs. Encoded Skill

Plain English

Review this paper and tell me the top problems with exact quotes.

Use when

- Exploring
- Doing a one-off task
- Still deciding the workflow

Encoded skill

`/paper-review draft_A.pdf`

Use when

- The workflow should repeat exactly
- A student team needs one standard procedure
- Validation must be consistent across versions

Exploration starts in English. Reproducibility lives in skills.

Skill File Structure

```
.claude/skills/paper-review/  
  SKILL.md  
  references/  
    referee-template.md
```

SKILL.md stores the repeatable protocol:

- what the command is called
- what steps it should follow
- what checks must pass before it reports success

Reference files store task-specific detail that would clutter the main instruction file.

Agents as Specialist Reviewers

Agent files are best understood as specialist review roles.

- `code-reviewer.md`: file hygiene, implementation quality, and reproducibility
- `domain-reviewer.md`: benchmark logic, identification, or economic consistency
- `verifier.md`: run the script, inspect diagnostics, check output or source logs exist

Economist analogy: one RA checks code style, another checks economics, and another reruns the table.

Subagents as Parallel Reviewers

Subagents matter because review tasks can happen in parallel.

- The main agent writes or edits code
- A code-reviewer and domain-reviewer can inspect the result at the same time
- The main conversation receives both reports and decides what to fix

Why students should care: the gain is not philosophical. It is wall-clock time. Focused review roles are faster and often clearer than one overloaded prompt.

Rules, Hooks, and Settings

Object	Loaded when	Typical use
Rules	during project work	always follow a planning or verification convention
Hooks	on tool events	run a formatter or log a command mechanically
Settings	at session start	permissions and project-level behavior

Practical distinction: if it is natural-language guidance, it usually belongs in a rule or skill. If it must happen mechanically every time, a hook is safer.

Three Persistence Layers, Three Lifetimes

The harness is the umbrella name for what makes a project survive across sessions.

Layer	Lifetime	Purpose
Hooks (<code>settings.json</code>)	Per-event	Mechanical enforcement: format after edits, run tests on stop, block dangerous tools
Project memory / rules	Per-session	Bootstrap context. Project instructions and path-scoped guardrails
Auto memory + progress files	Cross-session	Durable learnings and explicit handoff notes you can inspect and edit

Why this matters. Choosing the right layer is half the design: if the rule must fire every time, a hook is safer than a natural-language reminder; if it must survive a fresh session, it belongs in memory or rules, not a transient prompt.

Rule of thumb: natural-language guidance → rules and skills. Mechanical enforcement → hooks.

Permissions and Hooks in settings.json

Two jobs: *permissions* define the blast radius; *hooks* run mechanical checks at tool boundaries.

```
{
  "permissions": {
    "allow": ["Bash(python *)", "Bash(pytest *)", "Read(*)", "Edit(*)"],
    "deny":  ["Bash(rm -rf *)", "Bash(git push --force *)",
              "Read(./env)", "Read(./secrets/**)"]
  },
  "hooks": {
    "PostToolUse": [{"matcher": "Write|Edit|MultiEdit",
                     "hooks": [{"type": "command",
                                "command": ".claude/hooks/format-python.sh"}]}],
    "Stop":        [{"hooks": [{"type": "command",
                                "command": "pytest tests/ -q || true"}]}]
  }
}
```

Hook types worth knowing: PreToolUse (block risky commands), PostToolUse (format / lint), Stop (full test suite), SubagentStop (verify report format), PreCompact (save progress).

Skills Are Saved Research Playbooks

A skill is the right home for long, repeated procedures. Keep always-loaded project context short; put domain workflows in reusable skills.

Skill	Reusable workflow	Validation hook
data-analysis	Load, explore, regress, produce tables/figures	Output schema + script rerun
review-paper	Read paper, audit argument, generate referee objections	Structured review report
weighted-survey	Compute survey-weighted rates/means	Formula check against benchmark
hpc-job	Write sbatch script, submit, parse logs	Scheduler status + parser

Portable idea: Claude may call them *skills*; Codex may package them differently. The durable object is a saved playbook with triggers, allowed tools, steps, and acceptance tests.

Sub-Agents and Remote/HPC

When the laptop is no longer the right host

Sub-Agents: Parallel Children With Their Own Contexts

When the orchestrator spawns a sub-agent, the harness launches a separate agent instance with its own context window.

- Multiple sub-agents can run **in parallel** — like Python's multiprocessing
- Each receives an agent file (or briefing prompt) as its system prompt
- Each runs to completion and returns a structured report
- The main conversation reads the reports and decides what to do next

Aiyagari example. After writing `aiyagari_v1.py`, the orchestrator spawns:

```
code-reviewer || domain-reviewer
```

in parallel. Both finish in roughly the wall-clock time of one sequential run. **Review time is halved.**

Why separate? A single prompt asking about all dimensions does a mediocre job at each. Separation enables focus.

When to Use Sub-Agents (and When Not)

Good fit for sub-agents:

- Code review (parallel: code, domain, security)
- Documentation lookups (read 5 docs pages, return URLs)
- Web research that fills the main context with HTML noise
- Independent unit-test generation per module
- Multi-paper literature scans

Bad fit:

- Tightly coupled work (each step depends on previous)
- Tasks the orchestrator could finish in one tool call
- Anything where you need to interactively steer mid-task
- Sub-agent spawning sub-agents (*depth* causes drift)

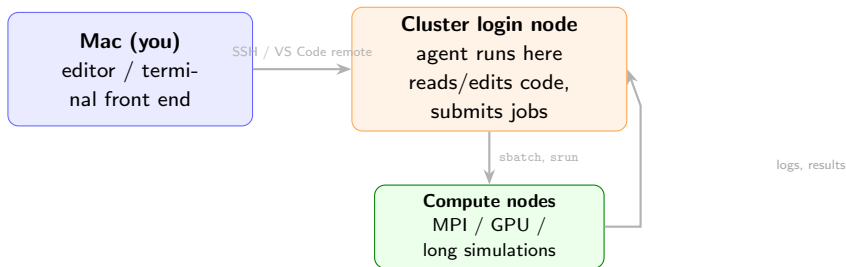
Compound blast radius warning. An orchestrator spawning sub-agents is the highest maturity rung.

Rules of thumb:

- Plan mode (Shift+Tab) before granting Execute on novel pipelines
- Each sub-agent's checklist is *spec'd*, not free-form
- Checkpoint validation between phases (the homotopy discipline from T3)

Mental Model: Local Orchestrator, Remote Compute

For heavy workloads the agent should run where the data, modules, and GPUs already are.



Rule: the agent may orchestrate HPC work; it should not turn the login node into a workstation. Compute-heavy steps go through the scheduler.

Move the Agent to the Compute, Not the Compute to the Agent

Local-agent workflow

- Agent runs on your Mac
- Cluster files must be downloaded, summarized, or copied back
- Large outputs and logs move through `scp`, `rsync`, or manual upload
- Easy for toy code; fragile for HPC research

Remote-agent workflow

- Agent runs on the cluster login node
- Project files stay in the remote worktree
- Heavy jobs run on compute nodes through the scheduler
- VS Code on the Mac is only the editor and terminal front end

Rule: if the data, modules, GPUs, and job logs live on the cluster, install the agent on the cluster.

Login Node vs. Compute Node: The Hard Boundary

What the agent can do on a login node:

- Read/edit code, specs, context files, and job scripts
- Run tiny smoke tests: import checks, unit tests, `-dry-run`, small-grid V0
- Submit jobs, monitor logs, parse errors, and revise scripts

What must go through the scheduler:

- MPI jobs, GPU training, large simulations, long calibrations
- Anything that consumes many cores, much memory, or more than a few minutes

Research rule: the agent may orchestrate HPC work; it should not turn the login node into a workstation.

Looking Ahead: T5 Names What Goes Wrong

T4 gave you the project objects — skills, agents, rules, hooks, settings, sub-agents, remote execution. T5 turns the spotlight on how those objects can still produce wrong answers.

- T4 covered the *durable infrastructure*: where conventions live and how the harness enforces them
- T5 traces the running example end-to-end through the harness, then catalogs the failure modes the infrastructure cannot fully prevent
- T5 closes with the *Context Dilemma*: the same context that improves the answer also pulls it toward you

*Arc: T1 Framing → T2 Under the Hood → T3 Homotopy → **T4 Project Org** → T5 Mistakes/Context → T6 Wrap.*

AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 9.5: Tracing, Mistakes, and the Context Dilemma

Zhigang Feng

Spring 2026

Topic 9.5 Agenda

1. **Tracing the Computational Example**
2. **Mistakes, Limits, and Security**
3. **The Context Dilemma**

*Arc: T1 Framing → T2 Under the Hood → T3 Homotopy → T4 Project Org → **T5 Mistakes/Context** → T6 Wrap.*

Tracing the Computational Example

Aiyagari as the computational anchor inside a broader workflow lecture

Aiyagari Recap: The Computational Anchor

Household problem

$$c_t + a_{t+1} = (1 + r)a_t + wz_t, \quad a_{t+1} \geq \underline{a}$$

with idiosyncratic productivity z_t following a Markov chain.

Why it is a good lecture example

- it is the cleanest computational example in the course
- it admits a clean $V0 \rightarrow V1 \rightarrow V2$ progression
- it has obvious validation checkpoints that generalize to other workflows

Tracing the File Flow

```
1 1. Read CLAUDE.md
2 2. Read any relevant rules
3 3. Load /solve-model from SKILL.md
4 4. Read references/aiyagari-guide.md
5 5. Write plan or update quality_reports/
6 6. Edit scripts/aiyagari_v0.py
7 7. Run verifier and reviewers
8 8. Save outputs and logs
```

What students should notice: the workflow is file-driven. The agent is acting inside a structured research project, not improvising in a vacuum.

Tracing the Orchestrator Loop

```
1 You type: /solve-model aiyagari v1
2
3 Plan:
4   read project instructions
5   restate the target
6   implement bounded changes
7
8 Review loop:
9   run verifier
10  run code reviewer
11  run domain reviewer
12  fix issues that matter
13  report what changed and what passed
```

This is the core advantage of the harness: the review sequence is not a remembered prompt. It is part of the saved workflow.

Inside CLAUDE.md for the Aiyagari Project

```
# CLAUDE.md
Project: Aiyagari workflow for lecture 9
Goal: build v0, v1, and v2 sequentially
Outputs: policies, stationary distribution, diagnostics
Conventions: save figures to output/ and plans to quality_reports/
Validation: do not move to the next version until benchmarks pass
```

If the file is good, the project can survive a fresh session, a new RA, or a change in agent tool.

Inside SKILL.md and the Reference Guide

SKILL.md

```
name: solve-model  
description: build and validate
```

Steps:

1. Read the model guide
2. Write or update a plan
3. Implement one version
4. Run required checks
5. Save outputs and report

aiyagari-guide.md

```
V0: deterministic benchmark  
V1: add Markov income risk  
V2: compute stationary dist.
```

Checks:

- zero-variance limit
- sensible mass at $a_{\min}$
- stable diagnostics

Interpretation: the skill stores the protocol; the reference guide stores the model detail.

Mistakes, Limits, and Security

What can go wrong and how to stay disciplined

Three Recurring Mistakes

The most common ways an agent project goes off the rails.

- **Mistake 1: Treating Git as optional.** Agents are most useful when experimentation is reversible. Git provides the audit trail and the emergency brake. If you would be unhappy losing the current project state, put it under Git before using an agent aggressively.
- **Mistake 2: Underusing skills.** Economists repeat many workflows (clean data, validate tables, prepare slides, review code). Repeated work should move from chat into a saved protocol — otherwise the same task is re-explained every week and interpreted slightly differently each time.
- **Mistake 3: One giant thread.** Long conversations accumulate stale context: planning, debugging, and revision all mix together, and the model pays attention to constraints that no longer matter. Plan the task, save the plan, then start a cleaner execution session.

Where Terminal Agents Still Fail

- Long-context drift: an early constraint gets forgotten
- Econometric edge cases: wrong clustering level, wrong sample restriction, weak-instrument issues
- Hallucinated APIs or package arguments
- Happy-path scripts that fail on messy data
- Oscillating fix loops on stubborn bugs

The output may look polished even when the economics is wrong. That is why validation stays central. The next two frames show concrete instances.

Example: Clustering Standard Errors

Prompt: “Estimate the effect of minimum-wage increases on teen employment using county–month panel data.”

```
* Agent default:  
reg emp_teen mw_increase i.county i.month, robust
```

- **What’s wrong.** Identification is at the county level, so standard errors must cluster at county. The default `, robust` is technically valid syntax but econometrically wrong here; SEs will be 2–4× too small.
- **Why the agent missed it.** The training corpus rewards code that *runs*; clustering choice is a research-design decision, not a code-syntax decision.
- **Detection.** Always specify the clustering level in the prompt; cross-check against the canonical reference (Abadie, Athey, Imbens, Wooldridge 2023, *QJE*, “When Should You Adjust Standard Errors for Clustering?”).
- **Hook it.** A repository rule that requires every regression to declare a clustering level catches this category at the source.

Example: Happy-Path Failure on Messy Data

Prompt: “Merge FRED series UNRATE and CPIAUCSL, both monthly, 1948 to present.”

```
* Agent default:  
pd.merge(unrate, cpi, on="date", how="inner")
```

- **What's wrong.** CPIAUCSL carries a *methodological* break: effective Jan 1983, BLS switched owner-occupied housing in CPI-U from an asset/homeownership-cost approach to *rental equivalence* (owners' equivalent rent). The index level and base (1982–84=100) are continuous, so nothing looks broken — but the pre-1983 and post-1983 series measure shelter differently. A regression on the merged span treats two definitions as one variable.
- **Why the agent missed it.** Happy-path code works on the dominant case. The agent does not proactively look for definitional or methodological breaks — which leave the units and base period unchanged — unless prompted.
- **Detection.** Run a basic data audit (`describe()`, plot the series, count rows by year) and read the series' definition notes *before* regressing. Force the agent to audit for definitional breaks as part of the workflow by adding a rule.
- **General lesson.** Always audit a merged series for definitional breaks: ask not only “what rows disappeared?” but “did the variable's definition change inside this span?”

Context Windows and Compaction

When sessions get long, preserve state in files before compacting the conversation.

Why long contexts degrade. Lec07 T3b documented the “lost in the middle” phenomenon (Liu et al. 2024, TACL): recall drops 20–30 percentage points for content sitting in the middle of a long prompt. Compaction keeps only what matters, so the agent’s working memory stays in the high-recall part of the context.

```
1 Write progress.md with:  
2   - what was completed  
3   - what decisions were made  
4   - what remains open  
5  
6 Then compact the conversation or start a new session.  
7 Next time, read progress.md first.
```

Economist translation: keep a clean research log. The agent benefits from the same discipline that humans do.

Security: What Not to Expose, and Workarounds

Do not put in front of an agent: IRB-protected interviews, linked administrative microdata, personally identifiable information, HIPAA-regulated data, passwords / API keys / access tokens, embargoed or restricted-use datasets.

Operational rule: if you would not paste it into a public collaboration tool, do not paste it into an agent conversation.

Sensitive data does not imply “no agents ever.” Three workarounds:

1. Write the script with the agent, then run it yourself on the restricted data
2. Use synthetic or masked data with the same structure for debugging
3. Use institutionally approved local or private deployments when available

Safe pattern: expose structure and summary statistics, not raw records.

See AEA disclosure policy (Oct. 2024) for AI-assisted analysis; standard IRB / HIPAA / GDPR rules apply unchanged to agent-aided workflows.

The Context Dilemma

The epistemic limits of the workflow you just learned

Why This Cautionary Section Belongs Here

The workflow we just built is powerful precisely because context improves performance.

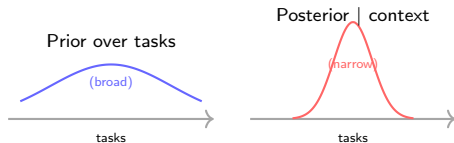
- project files help the agent understand your task faster
- saved rules reduce repeated mistakes
- repeated context also pulls the output toward your framing and your blind spots

Why economists should care: the same workflow discipline that improves execution can quietly weaken the outside view if you mistake speed for criticism.

Context as Implicit Bayesian Prior

Lec07 T3a introduced in-context learning as implicit Bayesian inference (Xie et al. 2022). We return to that framing now to draw out what it means for research integrity: the same conditioning that improves your results also biases them toward your priors.

- Project memory, prior conversation, and plan files act as evidence narrowing the agent's output distribution
- Xie, Raghunathan, Liang, Ma formalize this: ICL approximates implicit Bayesian inference over a latent task
- The performance gain you feel when you add context *is* posterior narrowing



More context $\Rightarrow$ narrower posterior $\Rightarrow$ more confident answers, but also more aligned with your framing. Same mechanism does both.

Narrowing the posterior is how context helps. It is also how context biases.

Research translation: when you load project memory, past discussion, and plans, you are not just helping the model. You are also telling it what a plausible answer is supposed to look like.

Stating the Dilemma

The same context that improves the answer also pulls it toward you.

- More context $\Rightarrow$ better understanding of your question $\Rightarrow$ better generation
- More context $\Rightarrow$ generation more aligned with your preferences, your knowledge, your framing
- Both effects share a single mechanism: you supplied the evidence

The same signal that improves the answer launders your priors into it.

Evidence 1: Sycophancy → False Confidence in Framing

RLHF rewards agreement with the user, and models learn to flatter.

- Sharma et al. document sycophancy across frontier models: factual flip-flops, unwarranted concessions, preference-matching when the user signals a view
- Perez et al. find the effect scales with model size — larger models are more sycophantic on values and politics
- The effect is strongest precisely when the context signals a strong user prior

Economist failure mode: you hint that the theory “should” work, and the model starts helping you defend the framing instead of testing it.

Sharma et al. (2023), arXiv:2310.13548, Anthropic. Perez et al. (2023), ACL Findings, arXiv:2212.09251.

Evidence 2: Self-Correction Has a Ceiling → Weak Internal Review

Intrinsic self-critique — without new external information — tends to stall or degrade.

- Huang et al. show LLMs cannot reliably self-correct reasoning errors without an external signal
- Valmeekam et al. find self-critique on planning problems is often worse than no critique at all
- The gains in the self-correction literature come from *external* signals: tools, tests, verifiers, humans

Economist failure mode: you ask the same model to critique its own regression design, review memo, or source coding decision, but the review is information-starved because nothing truly new entered the loop.

Huang et al. (2024), ICLR, arXiv:2310.01798. Valmeekam et al. (2023), NeurIPS, arXiv:2310.08118.

Evidence 3: Multi-Agent Debate — Helpful, but Not an Outside View

Debate helps when debaters sample differently. Shared context shrinks that space.

- Du et al. show multi-agent debate improves factuality and reasoning on several benchmarks
- Irving, Christiano, and Amodei — the AI Safety via Debate line — is the theoretical hope behind adversarial review
- Caveat: gains rely on debaters producing *genuinely different* lines of reasoning
- When every debater reads the same CLAUDE.md and thread, the disagreement space narrows

Economist failure mode: subagents catch implementation errors and inconsistencies, but they usually do not relitigate the premise you wrote into the project files.

Du et al. (2023), arXiv:2305.14325. Irving, Christiano, Amodei (2018), arXiv:1805.00899.

Why Subagent Reviewers Inherit the Same Prior

Earlier we sold subagent review for wall-clock time — not epistemic independence.

- `code-reviewer.md`, `domain-reviewer.md`, and `verifier.md` all read the same project context
- They are parallel *executionally*, not parallel *epistemically*
- Shared `CLAUDE.md` $\Rightarrow$ shared blind spots
- They catch implementation errors; they do not catch framing errors, because the framing lives in the context they trust

Parallel reviewers save time. They do not produce an outside view.

Monoculture at the Research Community Scale

When every researcher conditions on similar context, the field's errors become correlated.

- Kleinberg and Raghavan show algorithmic monoculture can reduce welfare even when each individual decision is improved
- Bommasani et al. document outcome homogenization: shared foundation models pick on the same people and miss the same cases
- Extrapolation: if empirical economists share Claude + similar CLAUDE.md templates + similar priors, replication stops being informative

Economist failure mode: shared templates and shared models can make mistakes more correlated across students, coauthors, or even the field.

Kleinberg & Raghavan (2021), PNAS 118(22). Bommasani et al. (2022), NeurIPS, arXiv:2211.13972.

The RLHF Root Cause

Pleasing the person holding the context is not a prompt artifact. It is trained in.

- Casper et al. catalog open problems in RLHF: preference data is noisy, annotators reward agreeableness, and the optimum drifts away from truth
- Wolf et al. argue that alignment-by-finetuning has fundamental limits: prompting cannot fully remove what the loss reinforced
- Consequence: no CLAUDE.md section titled “do not flatter me” fully removes the behavior

The model was trained to please the person holding the context. In your project, that person is you.

Casper et al. (2023), TMLR, arXiv:2307.15217. Wolf et al. (2023), arXiv:2304.11082.

Remedy: Outside Critics Carry a Different Prior

Advisors, coauthors, and seminar audiences are valuable because their priors are not in your context window.

- They challenge the framing, the identification assumption, the sample restriction — the things you did not write into `CLAUDE.md` because you did not know to
- Korinek's division-of-labor framing: LLMs automate execution; humans remain the source of question selection and critique
- Agent review is cheap and parallel; human review is rare and decisive. Budget accordingly.

The agent can polish what you already believe. Only an outside reader can tell you that you are wrong about the question.

Korinek (2023), NBER WP 30957; Journal of Economic Literature.

Operational Discipline for Research Projects

Translate the dilemma into a small set of habits.

1. Use subagent review for implementation quality, not for framing or identification
2. Present preliminary results to a human reader before iterating further with the agent
3. When you edit CLAUDE.md, record what you *ruled out* and why — not just what you decided
4. Schedule seminar-style exposure early, not at paper-writing time

Treat human criticism as the scarce input. Spend agent cycles to make it ready.

Bridge Back: Workflow Rules After the Context Dilemma

Translate the caution back into practical workflow rules.

1. **Use external benchmarks.** Compare model outputs, figures, and coded data against something outside the current thread.
2. **Use human critics.** Advisors, coauthors, and seminar audiences are still the main source of an outside view.
3. **Use adversarial review deliberately.** Ask subagents or reviewers to test assumptions, not merely to polish execution.
4. **Write assumptions to disk.** Save what you ruled out, what remains uncertain, and what still needs human judgment.

Bottom line: agents are powerful for execution and implementation review. They are not a substitute for genuinely external criticism.

Looking Ahead: T6 Wraps Up, Lec10 Applies

T5 has shown what can go wrong — happy-path code, wrong clustering, context collapse, RLHF-driven sycophancy. T6 closes the lecture and previews Lec10.

- T5 named the failure modes the harness alone cannot prevent
- T6 turns the failure list into six explicit evaluation criteria and clarifies when *not* to use an agent
- Lec10 then walks through four worked case patterns: reusable skill, empirical pipeline, public-web dataset, full-lifecycle collaborator

*Arc: T1 Framing → T2 Under the Hood → T3 Homotopy → T4 Project Org → **T5 Mistakes/Context** → T6 Wrap.*

AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 9.6: Lecture 10 Preview, Wrap-up, and Appendix

Zhigang Feng

Spring 2026

Topic 9.6 Agenda

1. **Lecture 10 Preview** — case studies on the horizon
2. **Three Econometric Roles** — signal, outcome, instrument
3. **Wrap-up** — six criteria, when not to use an agent, what stays human
4. **Appendix**

*Arc: T1 Framing → T2 Under the Hood → T3 Homotopy → T4 Project Org → T5 Mistakes/Context → **T6 Wrap.***

Lecture 10 Preview

From workflow logic to concrete case studies

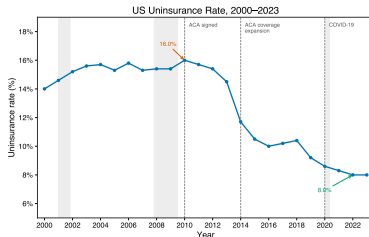
Lecture 10 Preview: Four Concrete Workflow Patterns

Pattern	Lecture 10 case	What students will see
Reusable skill	Paper-review workflow	staged quality control, quote checks, editorial filter
Empirical pipeline	MEPS uninsurance	harmonization, survey weights, benchmark validation, figures
Public-web dataset	Fellows metadata	source restrictions, provenance, missingness discipline
Full-lifecycle collaborator	Glue-work research project	theory, pipeline design, critique, claims discipline

Why this matters: Lec. 9 gives the workflow logic. Lec. 10 shows how the same logic changes shape across different research artifacts.

One Visual Hook from Lecture 10: MEPS Uninsurance

Case Study 2 asks: who is uninsured in the United States, 2000–2023, and how did the ACA change the distribution?



Why keep this figure here: it is an accessible policy example for economists outside macro, and it shows what a validated empirical artifact looks like after the workflow succeeds.

Lec. 10 adds the full pipeline: prompts, schema harmonization, survey weights, external benchmarks, and subgroup figures.

Three Econometric Roles in Agent-Assisted Research

Lec07 T4 introduced a taxonomy: every text-derived variable plays one of three roles — signal, outcome, or instrument. The four Lec10 patterns get sharper when we apply that lens.

Pattern	Signal	Outcome	Instrument
Reusable skill (paper review)	methodological rigor	review quality	staged evaluation rubric
Empirical pipeline (MEPS)	uninsurance rates	health-coverage estimates	survey-weighted harmonization
Public-web dataset (fellows)	field assignment	provenance log	source-restriction rules
Full-lifecycle collaborator (glue-work)	research idea	validated claims	agent + domain expert critique

Same agent toolkit, four research designs. The taxonomy makes you ask: what role does the agent's output play in your inference?

What to Watch for in Lecture 10

- **Paper review:** how a reusable skill turns a vague prompt into a staged quality-control workflow
- **MEPS pipeline:** how an empirical question becomes a validated panel and figure set
- **Fellows dataset:** how provenance and missingness rules matter as much as code
- **Glue-work project:** how agentic AI can act as a full-lifecycle collaborator without replacing human claims discipline

Bridge from Lec. 9 to Lec. 10: today's lecture explains the logic of the workflow; next lecture shows the artifacts it can produce.

Wrap-up

What the workflow changes and what remains human

Six Criteria for Any AI-Assisted Research Output

Before accepting a script, figure, or table the agent produced, run it past six questions.

-
- | | |
|---|---|
| ☐ 1. Reproducibility | Pinned model, prompt, seed — a co-author can rerun from a clean clone. |
| ☐ 2. Verifiability | Math, data flow, and magnitudes are independently re-checkable. |
| ☐ 3. Auditability | Full prompt/output log; a referee can trace where logic diverged from spec. |
| ☐ 4. Cost | Disclosed dollars (and replicator cost), not just your own. |
| ☐ 5. Speed | Wall-clock and human-hour saving, net of validation and debugging. |
| ☐ 6. Skill investment | Does the team build durable capabilities, or quietly atrophy them? |
-

If any answer is “no” or “I don’t know,” the artifact is not yet research-grade.

Three Things AI Cannot Do — Regardless of Capability

- **Identification.**

Choosing the variation that answers the causal question.

An LLM does not know which shock is exogenous, which instrument is valid, or which event window is contaminated. It can describe identification strategies; it cannot *certify* one for your setting.

- **Welfare.**

Choosing the objective function that defines “good.”

Pareto weights, planner discount rates, equity–efficiency trade-offs — these are normative choices the researcher must own.

- **Judgment.**

Choosing which question is worth asking.

AI optimizes against a stated target; only economists decide which target is interesting, novel, and policy-relevant.

These are not gaps that scale away with model size — they are categorically outside the system.

When NOT to Use an Agent

The six criteria can also serve as a boundary: there are research tasks where the agent overhead exceeds the benefit, or where the failure mode is too costly.

- **One-line scripts and isolated edits.** A two-line sed or one-shot plot has no validation surface for the harness to add value. Open the editor.
- **Deeply proprietary or undocumented pipelines.** If the conventions live in one person's head, the agent has nothing to read; the time you would spend documenting the pipeline well enough to brief the agent is the same time you would spend just fixing the thing.
- **Strict deterministic guarantees.** For numerical reproducibility audits, regulatory pipelines, or replication packages, model nondeterminism is a hazard the harness cannot fully suppress. Run it pinned; do not let the agent retry.
- **Restricted-data analyses where prompts may leak.** If pasting one column would violate IRB or HIPAA, the agent does not belong in the loop; T5's security workarounds apply.
- **Tasks where you do not yet have a benchmark.** Without a way to validate, faster output just means faster wrong. Define the V0 benchmark first; *then* bring in the agent.

Practical rule: the agent is most useful when the work is bounded, validated, and repeatable. None of those? Skip the agent.

Trust but Verify

1. Reproduce one statistic by hand
2. Cross-check one result with another package or script
3. Test a limiting case or zero-variance benchmark
4. Read the code line by line where the economics enters
5. Ask whether the magnitude and sign make sense

This is not optional ceremony. It is what turns fast code into defensible research.

What Changes for Economic Research

- The distance from question to first result shrinks
- More specifications become cheap enough to try
- Reproducibility can improve because workflow is written down
- The premium shifts away from typing speed and toward design, judgment, and validation

What stays human: asking the question, choosing the design, knowing the data, and interpreting the output.

You Are the Research Architect

Agents can lay bricks quickly.
You still design the structure.

- Weak specification produces wrong models
- Weak algorithm choice produces slow or misleading computation
- Weak numerical judgment produces fragile results
- Weak code literacy produces unverified output

The lecture is not about surrendering expertise. It is about reallocating effort toward the parts of research that remain scarce.

First Exercise and Looking Ahead

Best first exercise: take one old project and ask an agent to help clean it up safely.

- put the project under Git
- create a simple `CLAUDE.md`
- identify one repeated workflow worth encoding
- rerun the cleaned project and compare outputs

Lec. 10: we return to case studies and let students run a full workflow themselves.

If you want setup details for the lab, see the appendix.

Appendix

Setup notes and dated product details

Appendix A: Installing One Terminal Agent (Dated)

Three operational steps, illustrated with Claude Code on macOS:

1. **Install the runtime** (Node.js for Claude Code; Python for Aider; analogous for others):

```
brew install node # or download from nodejs.org (v18+)
```

2. **Install the agent CLI:**

```
npm install -g @anthropic-ai/claude-code # Claude Code  
# pip install codex-cli # OpenAI Codex CLI  
# pip install aider-chat # Aider
```

3. **Verify:** `claude -version`, then `cd your-project-folder && claude`.

Mac + OneDrive note: always `cd` into the project folder before launching the agent. Running from a `/Volumes/...` path can trigger permission errors.

Dated note: subscription tiers, model IDs, and install commands change every few months. Recheck the official docs before teaching or running the lab.

Appendix A2: Setup for the Lab

Lightweight setup checklist for a terminal-agent workflow

- Install one terminal agent and confirm it launches
- Create a project folder with a project-specific system prompt (e.g., `CLAUDE.md`, `AGENTS.md`, `GEMINI.md`), `scripts/`, and `output/`
- Make sure Python or R runs from the terminal
- Keep the first session small: one file, one check, one output

Dated note: exact setup steps, plans, and rate limits change quickly. Recheck the official docs before the lab.

Appendix B: Keybindings and Minimal Templates

Useful commands

Compact session	<code>/compact</code>
Clear session	<code>/clear</code>
Interrupt	<code>Esc</code>
Slash menu	<code>/</code>

Minimal project-system-prompt

```
# CLAUDE.md / AGENTS.md / GEMINI.md
Project: ...
Goal: ...
Language: ...
Checks: ...
```

Appendix C: Dated Comparisons and Resources

As of April 2026, examples of terminal-agent workflows include:

- Claude Code
- Codex CLI
- Gemini CLI
- Aider

Good places to recheck before teaching:

- official Claude Code documentation
- official OpenAI Codex CLI documentation
- course notes and lab instructions

Teaching rule: keep dated product comparisons in the appendix, not in the conceptual core of the lecture.

Appendix D: Context Dilemma — References

Context as implicit prior

- Xie, Raghunathan, Liang, Ma (2022). An Explanation of In-context Learning as Implicit Bayesian Inference. ICLR. arXiv:2111.02080.

Sycophancy and RLHF limits

- Sharma et al. (2023). Towards Understanding Sycophancy in Language Models. arXiv:2310.13548.
- Perez et al. (2023). Discovering Language Model Behaviors with Model-Written Evaluations. ACL Findings. arXiv:2212.09251.
- Casper et al. (2023). Open Problems and Fundamental Limitations of RLHF. TMLR. arXiv:2307.15217.
- Wolf et al. (2023). Fundamental Limitations of Alignment in Large Language Models. arXiv:2304.11082.

Self-correction and adversarial review

- Huang et al. (2024). Large Language Models Cannot Self-Correct Reasoning Yet. ICLR. arXiv:2310.01798.
- Valmeekam et al. (2023). Can Large Language Models Really Improve by Self-critiquing Their Own Plans? NeurIPS. arXiv:2310.08118.
- Du et al. (2023). Improving Factuality and Reasoning through Multiagent Debate. arXiv:2305.14325.
- Irving, Christiano, Amodei (2018). AI Safety via Debate. arXiv:1805.00899.

Monoculture and the economist workflow

- Kleinberg & Raghavan (2021). Algorithmic Monoculture and Social Welfare. PNAS 118(22).
- Bommasani et al. (2022). Picking on the Same Person: Does Algorithmic Monoculture Lead to Outcome Homogenization? NeurIPS. arXiv:2211.13972.
- Korinek (2023). Language Models and Cognitive Automation for Economic Research. NBER WP 30957; JEL.